

TD/TP 6 — Choisir un comportement selon l'objet réel

Classes abstraites et polymorphisme dynamique

MAM4 — Programmation C++ Durée : 2 heures

Objectif. Faire traiter des `Triangle` et des `Quadrilateral` par un même code écrit pour la classe de base `Element`. Le programme devra choisir `area()` selon l'objet réel, sans copier celui-ci et sans tester manuellement son type.

Les sources sont fournies dans `code-td-seance-06.zip`. Après extraction, compiler et exécuter avec :

```
g++ -std=c++20 -Wall -Wextra -pedantic \
    -fsanitize=address,undefined -g \
    element.cpp triangle.cpp quadrilateral.cpp report.cpp main.cpp \
    -o td6
./td6
```

Conserver les sanitizers pendant toute la séance. Les exercices 1 à 6 forment le parcours principal. Les dix dernières minutes consolident la différence entre référence et copie.

Exercice 1 — Le même objet, deux types à considérer (15 min)

Objectif concret. Distinguer le type écrit dans une déclaration du type réel de l'objet désigné.

Le programme initial contient :

```
const Triangle triangle{7, 2.5};

std::cout << triangle.area() << '\n';

const Element& element{triangle};
std::cout << element.area() << '\n';
```

Avant d'exécuter, répondre puis vérifier :

- Quel est le type statique de `element` ?
- Quel est son type dynamique, c'est-à-dire le type réel de l'objet désigné ?
- Pourquoi la première ligne affiche-t-elle 2.5, alors que l'appel par `element` affiche encore 0 ?
- La référence a-t-elle créé un nouvel objet `Element` ?

À retenir. Le type statique de `element` est `Element`. Son type dynamique est `Triangle`. Avec une fonction membre ordinaire, le type statique détermine encore la fonction appelée.

Exercice 2 — Faire dépendre l'appel de l'objet réel (20 min)

Objectif concret. Introduire `virtual` pour obtenir le comportement du `Triangle` à travers une référence de classe de base.

Dans `element.hpp`, modifier provisoirement l'opération :

```
class Element {
public:
    virtual double area() const;
    // ...
};
```

Conserver pour l'instant l'implémentation qui renvoie 0.0 dans `element.cpp`. Recompiler puis vérifier :

```
const Triangle triangle{7, 2.5};
const Element& element{triangle};

assert(element.area() == 2.5);
```

Faire le même test avec un `Quadrilateral{8, 4.0}`. Expliquer quelle implémentation est appelée dans chaque cas.

À retenir. Une fonction virtuelle peut être choisie à l'exécution selon le type dynamique de l'objet désigné par la référence ou le pointeur.

Exercice 3 — Exprimer une interface abstraite (20 min)

Objectif concret. Dire que tout élément possède une aire, sans inventer une aire par défaut pour un `Element` général. La valeur `0.0` de `Element::area()` est artificielle. Remplacer d'abord sa déclaration par une fonction virtuelle pure, en laissant encore le constructeur dans la partie publique :

```
virtual double area() const = 0;
```

Supprimer alors la définition de `Element::area()` dans `element.cpp`, puis essayer temporairement :

```
Element impossible{1};
```

Le diagnostic doit indiquer que `Element` est abstraite. Retirer cette ligne, puis placer le constructeur dans `protected`. La déclaration finale devient :

```
class Element {
public:
    int id() const;
    virtual double area() const = 0;

protected:
    explicit Element(int id);

private:
    int id_;
};
```

Vérifier ensuite que les classes concrètes restent constructibles :

```
Triangle triangle{7, 2.5};
Quadrilateral quadrilateral{8, 4.0};
```

Dans le constructeur de chaque classe dérivée, `Element{id}` appelle le constructeur de la classe de base pour construire le sous-objet `Element`. Le mot-clé `explicit` empêche une conversion implicite d'un entier vers le type construit ; il n'interdit pas cet appel direct.

Question. Pourquoi le constructeur de `Element` est-il accessible aux classes dérivées mais pas au reste du programme ?

À retenir. Une classe possédant une fonction virtuelle pure est abstraite : elle décrit un contrat commun, mais ne peut pas être instanciée directement.

Exercice 4 — Faire vérifier une redéfinition avec override (15 min)

Objectif concret. Utiliser le compilateur pour localiser immédiatement une signature incorrecte.

Ajouter `override` aux déclarations de `area()` dans `Triangle` et `Quadrilateral`. Provoquer ensuite volontairement l'erreur suivante dans `Triangle` :

```
class Triangle : public Element {
public:
    double area() override; // const oublié
    // ...
};
```

Compiler, lire le diagnostic, puis restaurer :

```
double area() const override;
```

Comparer les signatures `Element::area() const` et `Triangle::area()`. Sans `override`, la seconde ne redéfinit pas la fonction virtuelle pure : `Triangle` reste abstraite et l'erreur de compilation n'apparaît qu'à son instantiation.

À retenir. `override` transforme une erreur détectée à l'instanciation en un diagnostic local sur la déclaration fautive.

Exercice 5 — Détruire correctement à travers la classe de base (15 min)

Objectif concret. Comprendre pourquoi une classe de base polymorphe possède un destructeur virtuel.

Cette expérience utilise exceptionnellement `new/delete` pour isoler le problème. Ajouter temporairement une trace dans les destructeurs, puis tester :

```
Element* element{new Triangle{7, 2.5}};
delete element;
```

Avec un destructeur de base non virtuel, le compilateur peut avertir que la destruction à travers `Element*` est incorrecte et le comportement est indéfini. Corriger l'interface :

```
virtual ~Element() = default;
```

Retirer ensuite le test manuel et revenir aux objets locaux. Aucun `new/delete` ne doit rester dans le programme final.

À retenir. Si un objet peut être détruit à travers un pointeur de classe de base, le destructeur de cette classe de base doit être virtuel.

Exercice 6 — Traiter une collection hétérogène (15 min)

Objectif concret. Parcourir plusieurs types concrets avec un seul code client.

Créer deux objets locaux, puis une collection de pointeurs non propriétaires :

```
const Triangle triangle{7, 2.5};
const Quadrilateral quadrilateral{8, 4.0};

const std::vector<const Element*> elements{
    &triangle,
    &quadrilateral
};
```

Compléter et tester les fonctions suivantes :

```
void report(const Element& element);
double total_area(const std::vector<const Element*>& elements);
```

Le corps de `total_area` ne doit nommer ni `Triangle` ni `Quadrilateral`. Ajouter les tests :

```
assert(elements.size() == 2);
assert(elements[0]->area() == 2.5);
assert(elements[1]->area() == 4.0);
assert(total_area(elements) == 6.5);
```

Question. Le vecteur possède-t-il les deux objets ? Pourquoi leurs durées de vie doivent-elles couvrir tout le parcours du vecteur ?

À retenir. La collection est homogène en type de pointeur `const Element*`, mais les objets désignés sont de types différents. Ici, les pointeurs utilisent les objets locaux sans les posséder.

Exercice 7 — Référence, copie et slicing (10 min)

Objectif concret. Relier le polymorphisme dynamique à la distinction étudiée en séance 5.

Comparer :

```
const Element& element{triangle}; // même objet, type dérivé conservé
// Element copy{triangle};        // impossible : Element est abstraite
```

Répondre en une phrase à chaque question :

1. Pourquoi la référence permet-elle encore d'appeler `Triangle::area()` ?
2. Si la classe de base était concrète, que conserverait une copie par valeur vers cette base ?
3. Pourquoi `report` reçoit-elle donc `const Element&` ?

À retenir. Une référence de classe de base conserve l'objet dérivé. Une copie par valeur vers une base concrète créerait un nouvel objet de base et perdrait la partie dérivée : c'est le slicing.

Bilan — 5 min. Expliquer avec vos propres mots :

1. la différence entre type statique et type dynamique ;
2. le rôle de `virtual`, `= 0` et `override` ;
3. pourquoi le destructeur de `Element` est virtuel ;
4. pourquoi le programme final utilise références et pointeurs sans copier les objets.

À rendre. Les sources complétées, les tests exécutés sous sanitizers et les réponses courtes aux questions des exercices 1, 3, 6 et 7.